

ReceiptAI

QR digital receipts + photo uploads + a private, evidence-grounded AI purchase assistant

The problem: Receipts are scattered across paper, merchant apps and inboxes. Bank statements usually lack item-level details. People cannot easily find proof of purchase, compare old prices or calculate spending by product. The solution: Collect a receipt via post-purchase QR or photo/PDF upload; normalize both into one secure database; let users search and chat with citations.

MVP success criteria

A customer completes a simulated purchase, scans its QR on a phone and saves the digital receipt; then uploads and corrects a paper receipt. Both show in one private dashboard, and a chatbot answers exact and semantic questions using the saved purchases with supporting receipt links.

End-to-end architecture

MERCHANT QR	PHOTO / PDF
Simulated POS -> paid sale -> short-lived QR -> authenticated claim	Camera/upload -> private object storage -> async vision extraction -> review

Shared canonical pipeline: normalize -> deterministic financial validation -> duplicate check -> user confirmation where needed -> PostgreSQL receipt + line items -> background embeddings in pgvector.

Consumer layer: receipt history and downloads -> SQL analytics + metadata/keyword/vector retrieval -> LangGraph agents -> verified chat answer with sources.

Technical stack and scope

Layer	Use
App / API	Next.js + React + TypeScript; FastAPI + Pydantic; Supabase Auth
Data / search	PostgreSQL + pgvector; SQLAlchemy/Alembic; private S3-compatible files; Redis/RQ
AI / agents	Vision model + optional OCR; LangChain RAG; LangGraph; typed MCP tools; optional real A2A remote agent
Quality / operations	pytest, Playwright, GitHub Actions, OpenTelemetry/Langfuse, Docker, HTTPS cloud deployment

Out of scope in 30 days: real card processing, universal retailer integration, NFC hardware, native mobile applications, autonomous purchases and perfect OCR for every receipt layout.

Two receipt journeys, one source of truth

A. Merchant QR - Cashier creates and completes a simulated sale. - API verifies amounts and creates a random, expiring one-use claim. - Merchant displays QR; customer scans and previews. - Customer signs in and claims atomically. - Receipt becomes available in history even if embeddings are delayed.	B. Photo / PDF - Signed-in customer uploads JPG, PNG or PDF. - Backend validates file and saves original privately. - Worker extracts merchant/date/items/totals into typed JSON. - Rule engine checks arithmetic and likely duplicates. - Customer edits uncertain fields and confirms, then indexing runs.
---	---

Canonical data model

All completed receipts share the same structure. QR already has structured line items and must bypass OCR. Uploaded photos retain the original file, extraction history and user corrections. Money uses integer cents or Decimal, never floats.

Table	Key fields / responsibility
users / merchants	Account ID, role, merchant ownership and permissions.
sales / sale_items	Merchant, order ID, completed status, raw line items, quantities, tax, discounts and total.
receipt_claims	Sale ID, hashed QR token, expiry, claimed_by, claimed_at; unique and single-use.
receipts / receipt_items	Owner, source QR/PHOTO, date, merchant, status, normalized and raw product names, amount.
receipt_files / extraction_runs	Private file key/hash; extraction model/version, validation errors and correction history.
receipt_embeddings / chat	Owner-scoped vector+text metadata; chat sessions, evidence IDs and messages.

Core API contracts

Endpoint group	Implement
Merchant / QR	POST /merchant/sales; POST /merchant/sales/{id}/complete; GET /public/receipts/{token}; POST /receipts/claim.
Photo / review	POST /receipts/upload; GET /receipts/{id}/status; PATCH /receipts/{id}/draft; POST /receipts/{id}/confirm.
History / assistant	GET /me/receipts; GET /me/receipts/{id}; GET /me/spending; POST /chat.
Integration (later)	POST /webhooks/pos/{provider}; verify signature and deduplicate event ID.

Non-negotiable controls

QR: cryptographically random token, server-stored hash, short expiry, atomic claim, replay protection; a publicly displayed QR can be scanned by the wrong person, so limit preview and add buyer confirmation for a real pilot. Upload: file-content checks, size/page limits, private storage, no public source images. Everywhere: owner-scoped SQL, vectors, files and MCP calls; typed responses; audit records; delete/export controls.

30-day build roadmap / Part I

Stages 1-4 establish reliable ingestion. Each numbered item is an implementation task with an explicit result and test.

STAGE 1 | Days 1-2 | Requirements & test fixtures

01.01 Define QR, photo, and merchant user journeys plus failure states.
01.02 Write MVP scope and measurable acceptance criteria.

01.03 Create synthetic/consented images, sales, edge cases and expected outputs.
01.04 Draw architecture and threat model: tokens, uploads, user isolation.

Skills: PRD, user stories, test design, system/security modeling
Outcome: Journeys, versioned fixtures, architecture diagram.
Done when: Every journey has a success/failure test and usable seed data.

STAGE 2 | Days 3-5 | Backend, authentication & schema

02.01 Initialize Next.js, FastAPI, linting and Docker Compose.
02.02 Create PostgreSQL models, constraints and Alembic migrations.
02.03 Implement verified customer/merchant login and role checks.

02.04 Define strict canonical receipt and financial amount schemas.
02.05 Add private object storage and authorized file reads.

Skills: React, TypeScript, FastAPI, SQL, Pydantic, auth, Docker
Outcome: Working application skeleton and secure shared database.
Done when: Fresh clone starts locally; second user cannot access first user data.

STAGE 3 | Days 6-9 | QR sale -> scan -> claim

03.01 Build merchant item catalog and simulated checkout UI.
03.02 Persist completed sales; recalculate all basket amounts.
03.03 Create expiring one-use receipt claim and QR link.

03.04 Build responsive guest receipt preview for phone camera scans.
03.05 Implement authenticated atomic claim and owned receipt record.
03.06 Handle expired, replayed and reissued claim attempts.

Skills: Checkout UX, SQL transactions, secure tokens, QR, concurrency
Outcome: One real phone can scan and save a simulated merchant purchase.
Done when: Known basket matches stored totals; concurrent/double claims fail safely.

STAGE 4 | Days 10-14 | Photo / PDF extraction & review

04.01 Build mobile camera, JPG/PNG/PDF upload and preview.
04.02 Validate upload and save original; enqueue processing job.
04.03 Preprocess source; extract typed merchant, date, items and totals.
04.04 Preserve raw product text while normalizing names/categories.

04.05 Check sums, discounts, dates, missing values and confidence.
04.06 Detect possible duplicate files and QR/photo duplicate purchases.
04.07 Show editable extraction beside source; confirm final receipt.

Skills: React uploads, private storage, Redis/RQ, vision/OCR, Pydantic, validation
Outcome: Reviewed paper receipts enter exactly the same database as QR receipts.
Done when: Blurry/incorrect receipts request correction; edited values survive refresh.

30-day build roadmap / Part II

Stages 5-8 turn the receipt data into a useful, evaluated, deployed AI product.

STAGE 5 | Days 15-17 | Unified customer dashboard

05.01 Build paginated history mixing QR and PHOTO records.
05.02 Add merchant/date/category/item search and filters.
05.03 Implement server-side spending totals and product price history.

05.04 Recompute views and invalidate indexes after corrections.
05.05 Add receipt download, data export and deletion controls.

Skills: React state, PostgreSQL indexes, SQL aggregation, privacy UX
Outcome: Useful receipt history and trustworthy exact financial metrics.
Done when: One QR + one photo display together; fixture totals match dashboard.

STAGE 6 | Days 18-22 | RAG, chatbot, MCP & agents

06.01 Embed confirmed receipt summaries/items with owner metadata.
06.02 Build user-scoped hybrid keyword + pgvector retrieval.
06.03 Expose typed, authorized MCP receipt and analytics tools.
06.04 Build LangGraph supervisor with persistent conversation state.

06.05 Implement research and SQL spending-analysis agents.
06.06 Add answer verifier, receipt citations and mobile chat UI.
06.07 Create 100-question mixed QR/photo evaluation benchmark.

Skills: LangChain, pgvector, hybrid RAG, MCP, LangGraph, tool calling, evals
Outcome: Chat answers exact and semantic purchase questions with evidence.
Done when: Cross-account retrieval blocked; numeric responses match SQL ground truth.

STAGE 7 | Days 23-26 | Reliability, security & interoperability

07.01 Add idempotent jobs, bounded retries, backoff and failure queue.
07.02 Optionally deploy one genuine A2A photo processor with Agent Card.
07.03 Build signed simulated POS webhook with event deduplication.

07.04 Instrument traces, model versions, error states, time and cost.
07.05 Test API/MCP/file/vector account isolation and prompt injection.
07.06 Run integration and Playwright end-to-end failure scenarios.

Skills: Redis/RQ, resilience, A2A, webhooks, AppSec, tracing, Playwright
Outcome: Traceable workflows that survive failures and isolate users.
Done when: Model/worker outage retains uploaded file; forged webhooks rejected.

STAGE 8 | Days 27-30 | Test, deploy & publish

08.01 Package backend, frontend, worker and DB migration commands.
08.02 Configure GitHub Actions: lint, tests, build and deploy gates.
08.03 Deploy HTTPS web/API, PostgreSQL+pgvector, Redis and private files.
08.04 Run extraction, RAG, financial and performance benchmarks.

08.05 Run release-blocking replay, isolation and adversarial tests.
08.06 Write backup, restore, deletion and failed-job recovery runbook.
08.07 Create README, diagrams, API docs and recorded full-flow demo.
08.08 Collect pilot feedback and document the release limitations.

Skills: Docker, CI/CD, cloud operations, pytest, metrics, technical writing
Outcome: Working public demo and honest reproducible engineering report.
Done when: Phone scans deployed QR; photo upload and chat work; launch tests pass.

Agent design, tests & deployment gates

Agent boundaries and the correct tool for each question

Component	How it works / outcome
Photo extraction agent	Vision model creates typed receipt draft; deterministic validator detects mismatches; user confirms uncertainties.
Supervisor agent	LangGraph selects exact SQL, semantic search or mixed workflow; persists state and routes failures.
Research agent	LangChain hybrid RAG over user-owned indexed items and receipts; returns IDs and source text.
Spending agent	Uses predefined, parameterized SQL functions for totals, categories, price changes and purchase counts.
Verification agent	Checks totals against tool output; verifies citations and account ownership; says when evidence is missing.
MCP / A2A	MCP server exposes typed receipt tools with caller authorization; optional independent photo processor exchanges genuine A2A tasks.

Test pyramid and measurable targets

Test layer	Critical cases / measurement
Unit + integration	Totals, discounts, date parsing, claim expiry, concurrent ownership, migrations, storage, worker retries, MCP contract.
End-to-end	Merchant completes sale -> phone QR -> claim; phone photo -> review -> confirm; both -> chat; duplicate and expiry flows.
RAG + agent evaluation	Labeled receipt photos and 100 questions; field/item accuracy, Recall@5, answer correctness, citation correctness, agent/tool routing.
Security + reliability	Two-account isolation across DB/vectors/files/MCP/chat; replay, malformed files, forged webhooks, injected receipt text, failover.
Operations	Track P50/P95 acquisition/chat latency, extraction time, error/retry rate, model version, token and dollar cost.

Deployment checklist

Infrastructure: Dockerized Next.js + FastAPI + worker; managed PostgreSQL with pgvector; Redis job queue; private object bucket; HTTPS; secrets from hosting environment; automated migrations. CI/CD: GitHub Actions executes lint, type checks, pytest, integration tests, Playwright smoke and image build before deployment. Runbook: health checks, structured logs, trace links, backups and verified restore, failed-job reprocessing, rollback and complete account deletion.

Final acceptance demonstration

On two devices: (1) cashier creates a known three-item sale; (2) customer scans hosted QR and saves it; (3) customer uploads a paper receipt and corrects one field; (4) both appear in the same account; (5) chatbot finds a named product and calculates combined spending with linked receipts; (6) another account cannot access any of that data; (7) forced worker failure recovers without losing either receipt. Publish measured results, not invented scores.

Resume story: Secure merchant-to-consumer QR acquisition + multimodal document AI + unified financial records + hybrid RAG + LangGraph/MCP orchestration + deterministic verification + production testing/CI/CD and deployed full-stack application.